SAN JOSÉ STATE UNIVERSITY · CMPE 172 ENTERPRISE SOFTWARE PLATFORMS

TutorBook

Term Project, Milestone 2 Report

Core Implementation: Booking & Concurrency

Student	Samuel Dinkayehu

Scenario	Tutoring center appointment booking
Stack	Java 17+ · Spring Boot 4 · Spring Security · SQL over JDBC (no ORM) · Thymeleaf · H2
GitHub	https://github.com/samuel1gg/cmpe172-tutorbook
Date	October 8, 2026

Contents

1. What Milestone 2 delivers
2. Frontend-to-backend request flow
3. The booking flow
4. Concurrency and transactions: race condition, ACID, isolation level, optimistic locking, retry, backstop
5. Login and role-based access control
6. Enterprise behavior: validation and global exception handling
7. Testing
8. Screenshots
9. How to run

1. What Milestone 2 Delivers

#	Requirement	Where it is implemented
1, 6	Customer and provider log in (BCrypt, server-side session, role read via JDBC)	`SecurityConfig`, `JdbcUserDetailsService`, `UserRepository`
2	Browse, filter (provider, service, date) and paginate slots with SQL `LIMIT` / `OFFSET`	`SlotController` → `SlotService` → `SlotRepository.findAvailable`
3	Book an available slot for a chosen service	`POST /slots/{id}/book` → `BookingService` → `BookingTransaction`
4	View my appointments (upcoming + history)	`GET /my/appointments` → `AppointmentService.getCustomerAppointments`
5	Cancel my appointment (owner-only)	`POST /my/appointments/{id}/cancel` → `AppointmentService.cancel`
7	Provider: create / remove availability slots tied to a service	`ProviderController` → `ProviderSlotService`
8	Provider: view appointments booked with me	`GET /provider/appointments`
	Statuses BOOKED, CANCELLED, COMPLETED	CHECK constraint + `AppointmentStatusJob` (marks past BOOKED appointments COMPLETED)
	Concurrency control + transactions + retry + unique backstop	Section 4
	Validation and global exception handling (400/403/404/409, no stack traces)	`BookingForm`, `SlotForm`, `GlobalExceptionHandler`, `ApiExceptionHandler`

	Unit tests and a two-thread concurrency test	61 tests, all passing (Section 7)

Frontend choice: server-rendered **Thymeleaf** HTML pages (the Page Controller pattern, as chosen in Milestone 1). Each page has its own controller method, and forms are plain HTML POSTs protected by Spring Security's CSRF token.

2. Frontend-to-Backend Request Flow

This traces what happens when a logged-in student clicks **Confirm booking**, i.e. `POST /slots/7/book`, from the browser down to the database and back.

```
Browser form —POST /slots/7/book (+ JSESSIONID cookie, CSRF token, notes)→
  Spring Security filter chain    session → AppUser; checks CSRF; checks hasRole("CUSTOMER")
(else 403)
  DispatcherServlet               routes to SlotController.book(...)
  Controller   SlotController     binds + @Valid BookingForm (else re-show form, 400)
  Service      BookingService     retry loop (max 3)  ⟶  BookingTransaction.tryBook
@Transactional
  Repository   SlotRepository / AppointmentRepository   SQL via JdbcClient
  Database     H2                 SELECT slot · UPDATE version · INSERT appointment · COMMIT
← 302 Location: /appointments/{id}/confirmation   (or 409 / 404 page from
GlobalExceptionHandler)
Browser follows redirect —GET→ AppointmentController → AppointmentService →
AppointmentRepository
← Thymeleaf renders confirmation.html
```

1. **UI (Thymeleaf form).** `book.html` renders a form with `th:action="@{/slots/{id}/book}"`. Thymeleaf automatically adds the hidden `_csrf` token.
2. **Security filters.** Before any controller runs, Spring Security loads the `AppUser` from the HTTP session, verifies the CSRF token, and applies the URL rule `/slots/*/book → hasRole("CUSTOMER")`. A provider gets 403 here and an anonymous user is redirected to `/login`.
3. **Controller.** `SlotController.book` receives the slot id from the path, the validated `BookingForm`, and the logged-in user through `@AuthenticationPrincipal`. It contains no business logic. It calls `bookingService.book(user.userId(), slotId, notes)` and redirects to the confirmation page (Post/Redirect/Get, so refreshing does not book twice).
4. **Service.** `BookingService` owns the retry policy, and `BookingTransaction` owns the rules and the transaction (Section 4).
5. **Repository.** Hand-written SQL through Spring's `JdbcClient`, using named parameters and explicit row mappers.
6. **Database.** H2 executes the statements inside one transaction and enforces the constraints.
7. **Back up.** The new appointment id comes back up the stack, the browser follows the redirect, and the confirmation page is rendered. If anything fails, the exception propagates to `GlobalExceptionHandler`, which renders `error.html` with the right status code.

3. The Booking Flow

3.1 Controller

```
@PostMapping("/slots/{slotId}/book")
public String book(@PathVariable long slotId,
                   @Valid @ModelAttribute("bookingForm") BookingForm form,
                   BindingResult errors,
                   @AuthenticationPrincipal AppUser user,
                   Model model, HttpServletResponse response) {
    // invalid input (e.g. notes over 500 chars): show the form again with 400
    if (errors.hasErrors()) {
        response.setStatus(HttpStatus.BAD_REQUEST.value());
        model.addAttribute("slot", slotService.getAvailableSlot(slotId));
        return "book";
    }
    long appointmentId = bookingService.book(user.userId(), slotId, form.notesOrNull());
    return "redirect:/appointments/" + appointmentId + "/confirmation";
}
```

3.2 Service: retry loop

```
public long book(long customerId, long slotId, String notes) {
    for (int attempt = 1; ; attempt++) {
        try {
            // each attempt is its own transaction
            return bookingTransaction.tryBook(customerId, slotId, notes);
        } catch (OptimisticConflictException e) {
            // the version changed under us: retry, up to MAX_ATTEMPTS (3)
            if (attempt >= MAX_ATTEMPTS) {
                throw new SlotConflictException("This slot is in high demand right now. Please
try again.");
            }
        }
    }
}
```

3.3 One attempt = one transaction

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public long tryBook(long customerId, long slotId, String notes) {
    // 1. read the slot and its version (404 if missing or removed)
    SlotState slot = slotRepository.findState(slotId)
            .filter(s -> !s.removed())
            .orElseThrow(() -> new NotFoundException("Slot " + slotId + " does not exist"));

    // 2. business rules (409)
    if (!slot.startTime().isAfter(LocalDateTime.now(clock))) {
        throw new SlotConflictException("This slot has already started ...");
    }
    if (slot.booked()) {
        throw new SlotConflictException("Sorry, this slot was just booked by someone else.");
    }

    // 3. optimistic version check: 0 rows updated means someone else won -> rollback + retry
    if (!slotRepository.bumpVersion(slotId, slot.version())) {
        throw new OptimisticConflictException(slotId);
    }

    // 4. insert; the UNIQUE(active_slot_id) constraint is the backstop
    try {
        return appointmentRepository.insertBooked(slotId, customerId, notes);
    } catch (DuplicateKeyException e) {
        throw new SlotConflictException("Sorry, this slot was just booked by someone else.");
    }
}
```

`BookingTransaction` is a separate Spring bean from `BookingService` for two reasons. First, `@Transactional` works through a proxy, and a method calling another method in the same class would bypass that proxy. Second, it guarantees that **each retry runs in a brand-new transaction** that sees freshly committed data.

3.4 The SQL

```
-- 1. read the slot, its version, and whether it already has an active booking
SELECT s.slot_id, s.provider_id, s.start_time, s.version,
       s.removed_at IS NOT NULL AS removed,
       EXISTS (SELECT 1 FROM appointments a WHERE a.active_slot_id = s.slot_id) AS booked
FROM availability_slots s WHERE s.slot_id = :slotId;

-- 3. optimistic version check: succeeds for exactly one concurrent transaction
UPDATE availability_slots SET version = version + 1
WHERE slot_id = :slotId AND version = :expectedVersion;     -- 1 row = we won, 0 rows = retry

-- 4. create the booking
INSERT INTO appointments (slot_id, customer_id, status, notes)
VALUES (:slotId, :customerId, 'BOOKED', :notes);
```

3.5 Cancel (owner-only)

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void cancel(AppUser user, long appointmentId) {
    AppointmentDetails a = find(appointmentId);              // 404 if missing
    if (a.customerId() != user.userId())
        throw new ForbiddenException("You can only cancel your own appointments.");   // 403
    if (!"BOOKED".equals(a.status()))
        throw new SlotConflictException("This appointment is already ...");          // 409
    if (!a.startTime().isAfter(LocalDateTime.now(clock)))
        throw new SlotConflictException("... has already started ...");              // 409
    if (appointmentRepository.cancel(appointmentId) == 0)
        throw new SlotConflictException("This appointment was already cancelled.");  // 409
}

-- compare-and-set: if two cancels race, only one changes a row
UPDATE appointments SET status = 'CANCELLED', cancelled_at = CURRENT_TIMESTAMP
WHERE appointment_id = :id AND status = 'BOOKED'
```

Cancelling sets `active_slot_id` to NULL (it is a generated column), so the slot immediately shows up as available again while the cancelled row stays in the customer's history.

3.6 Provider slot management

Create. The provider picks a service, a date and a start time. The end time is computed as *start + the service's duration*. The service rejects a start time in the past (400) and an overlap with another of the provider's slots (409), using `start_time < :end AND end_time > :start`. A `UNIQUE(provider_id, active_start_time)` constraint is the database backstop.

Remove. Only the owning provider (403 otherwise), and only if nobody has booked the slot (409 otherwise). Removal is a soft delete (`removed_at`), so cancelled appointments that reference the slot are preserved. Removal also performs the **same version bump** as booking, so "provider removes slot" and "student books slot" cannot both succeed at the same moment.

3.7 COMPLETED status

`AppointmentStatusJob` runs at startup and every minute (`@Scheduled`). It executes `UPDATE appointments SET status='COMPLETED' WHERE status='BOOKED' AND slot_id IN (SELECT slot_id FROM availability_slots WHERE end_time <= :now)`. The CHECK constraint now allows `BOOKED`, `CANCELLED` and `COMPLETED`.

4. Concurrency and Transactions

4.1 The race condition

Slot 7 (James Kim, Java Programming Help, Saturday 9:00 AM) is open. Alice and Bob both have the booking page open and click **Confirm booking** within the same millisecond. Two request threads now run the same code at the same time. Without concurrency control:

1. T1 (Alice) reads slot 7: not booked.
2. T2 (Bob) reads slot 7: *also* not booked, because T1 has not inserted anything yet.
3. T1 inserts Alice's appointment and commits.
4. T2 inserts Bob's appointment and commits.

Both students receive a confirmation for the same tutor at the same time. This is a classic **check-then-act race**, also called a lost update: both transactions made their decision from a read that became stale before they wrote.

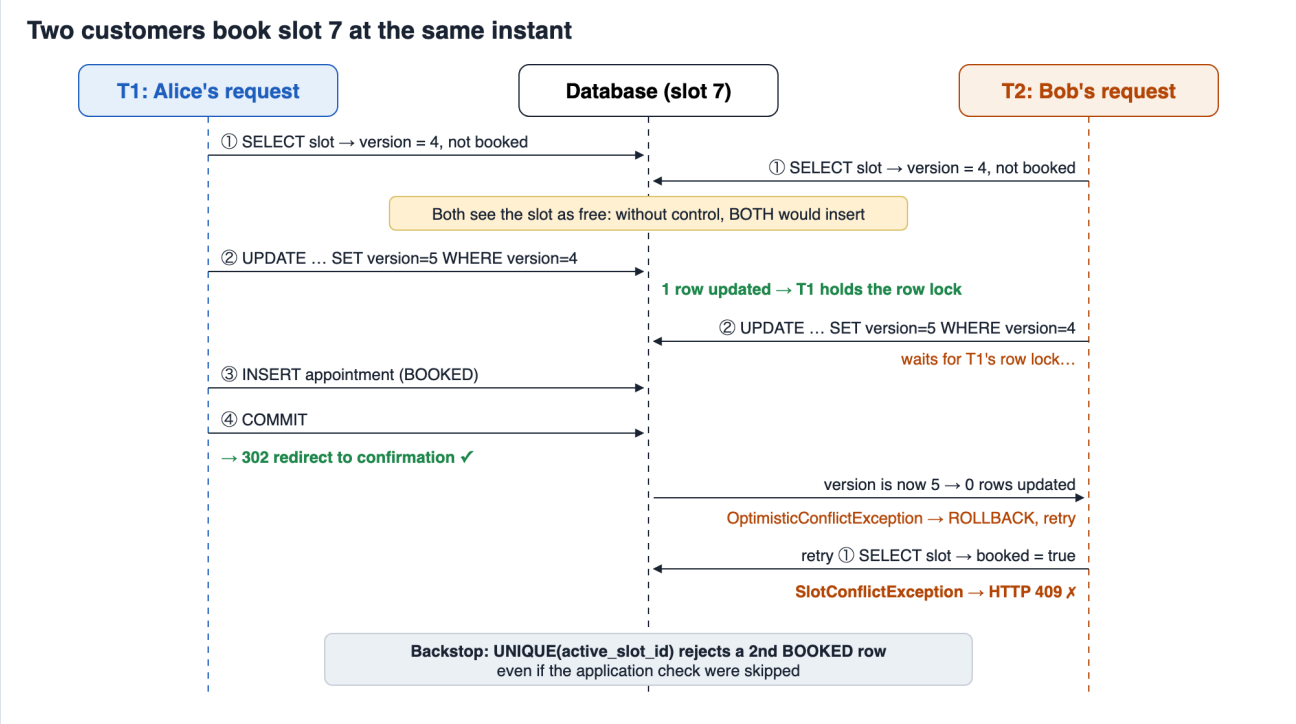


Figure 1. How TutorBook resolves the race: the version check lets exactly one transaction claim the slot; the other rolls back, retries, sees the slot is booked, and returns 409.

4.2 ACID applied to booking

Property	What it means for a booking	How TutorBook gets it
Atomicity	The version bump and the appointment insert happen together or not at all. A failed attempt leaves no half-claimed slot.	`@Transactional` on `tryBook`; any RuntimeException (including `OptimisticConflictException` and `DuplicateKeyException`) rolls back both statements.
Consistency	The database never contains two BOOKED appointments for one slot, a slot whose end is before its start, or an unknown status.	Constraints enforced by the database: `UNIQUE(active_slot_id)`, `CHECK(end_time > start_time)`, `CHECK(status IN …)`, foreign keys.
Isolation	Concurrent bookings must not see or overwrite each other's uncommitted work.	`READ_COMMITTED`, plus the version check, which turns the competing writes into a single winner (4.3–4.4).
Durability	Once the student sees the confirmation, the booking survives a crash.	The redirect to the confirmation page happens only *after* the transaction commits. With a production database (MySQL/PostgreSQL), a commit is written to the write-ahead log before it returns. The demo uses in-memory H2, which is durable only for the life of the process.

4.3 Isolation level: READ COMMITTED

The booking transaction runs at **READ COMMITTED**: a transaction only ever reads data that other transactions have committed, so there are no dirty reads.

- **Why it is enough here.** READ COMMITTED alone does *not* stop the race in 4.1, because both transactions can read "not booked". The protection comes from the next step. The `UPDATE … WHERE version = :expected` is a single atomic statement that takes a row lock. The second writer waits for the first to commit and then re-evaluates the `WHERE` against the *committed* row, so its condition no longer matches and 0 rows are updated. Combined with the version check, READ COMMITTED gives the guarantee we need.
- **Why not SERIALIZABLE.** SERIALIZABLE would also prevent the anomaly, but it makes the database abort transactions much more aggressively, including ones that touch *different* slots on some engines. That costs throughput, needs the same retry logic anyway, and behaves differently across databases. We only need to serialize writes to one row, and the version column does exactly that.
- **Why not REPEATABLE READ.** It would not help with the core problem: two transactions can still both read "not booked" from their snapshots. It also adds no benefit over READ COMMITTED plus the version check for this short transaction.
- READ COMMITTED is also the default for H2 and PostgreSQL, so the explicit annotation documents the intent rather than changing behavior.

4.4 Concurrency control: optimistic version check

Each `availability_slots` row has a `version INT` column. A transaction that wants to book (or remove) a slot:

1. reads the slot together with its `version`;
2. checks the rules in Java;
3. executes `UPDATE … SET version = version + 1 WHERE slot_id = ? AND version = ?`.

If another transaction changed the slot in between, the version no longer matches, **0 rows** are updated, and the attempt fails with `OptimisticConflictException` and rolls back. Only one of any number of concurrent transactions can move the version from 4 to 5.

Why optimistic rather than pessimistic (`SELECT … FOR UPDATE`)?

- **Conflicts are rare.** Most bookings are for different slots. Optimistic locking costs nothing when there is no conflict, and only the rare loser pays with one retry. A pessimistic lock would be taken on every booking, even uncontended ones.
- **Short locks, no deadlock risk.** The only lock held is the implicit row lock of a single UPDATE, held until a commit a few milliseconds later. No lock is held while Java code runs between the read and the write.
- **Portable.** It is plain SQL that works the same on H2 (used here), MySQL, PostgreSQL and SQLite. The assignment notes that `FOR UPDATE` is not available on every database.
- **It also protects other writers.** Provider slot removal uses the same version bump, so a removal and a booking of the same slot are serialized too.

Trade-off: under very heavy contention for *one* slot, many transactions do work that is then rolled back. For a tutoring center this is acceptable, and the retry cap bounds it.

4.5 Retry

A failed version check does not necessarily mean the slot is gone. It only means *something* changed it. So `BookingService` retries up to **3 times**, each time in a **new** transaction that re-reads the committed state:

- If the other transaction booked the slot, the retry sees `booked = true` and fails fast with **409**. This is not retried, because it is a real conflict.
- If the change was harmless (for example a version bump from a failed attempt that rolled back), the retry succeeds.
- After 3 version failures the user gets 409 "in high demand, please try again" instead of an endless loop.

Only `OptimisticConflictException` is retried. Business-rule failures (404, 409, 403) are never retried.

4.6 Database-level backstop

```
active_slot_id BIGINT GENERATED ALWAYS AS (CASE WHEN status = 'BOOKED' THEN slot_id ELSE NULL
END),
CONSTRAINT uq_appt_active_slot UNIQUE (active_slot_id)
```

The unique constraint from Milestone 1 is kept as the last line of defence. If any code path ever skipped the version check, for example a future admin tool or a bug, the database would still reject a second BOOKED row for the same slot with a `DuplicateKeyException`, which `tryBook` converts to 409. `DoubleBookingGuardTest` verifies this by inserting directly through the repository and bypassing the service.

4.7 Evidence

`ConcurrentBookingTest` starts two threads, makes both wait on a `CountDownLatch` "starting gun", then releases them together to book the same slot. It asserts that **exactly one succeeds**, the other gets `SlotConflictException`, and the database has exactly one BOOKED row. It runs for five different slots, plus a heavier case with **8 threads on one slot** (exactly 1 success, 7 conflicts). The application log from a test run shows the optimistic check doing the work:

```
[pool-3-thread-1] BookingService : Version conflict booking slot 1 (attempt 1/3)
[pool-4-thread-2] BookingService : Version conflict booking slot 3 (attempt 1/3)
...
[pool-8-thread-4] BookingService : Version conflict booking slot 11 (attempt 1/3)
[pool-8-thread-6] BookingService : Version conflict booking slot 11 (attempt 1/3)
Tests run: 6, Failures: 0, Errors: 0
```

In each race the loser hit the version conflict, retried, saw the slot was booked, and received 409. The test passes on every run.

5. Login and Role-Based Access Control

5.1 Login: BCrypt and a server-side session

- **Passwords** are stored only as BCrypt hashes, for example `$2a$10$Lg17hpQnSXpl…`. BCrypt is salted (identical passwords get different hashes) and deliberately slow, which resists brute force. `SecurityRbacTest` asserts that every `password_hash` starts with `$2a$10$` and that none contains a plaintext password.
- **Login** is a form POST to `/login`. Spring Security calls `JdbcUserDetailsService.loadUserByUsername`, which reads the user's row **over JDBC** (email, hash, role, and provider_id for tutors). Spring Security then checks the typed password with `BCryptPasswordEncoder.matches`.
- **Session:** on success, the `AppUser` (including its role from the database) is stored in the **server-side HTTP session**. The browser only holds the opaque `JSESSIONID` cookie. Controllers receive the user through `@AuthenticationPrincipal AppUser user`, so the client can never claim a different identity or role.

```
SELECT u.user_id, u.email, u.password_hash, u.full_name, u.role, p.provider_id
FROM users u LEFT JOIN providers p ON p.user_id = u.user_id
WHERE LOWER(u.email) = LOWER(:email)

// AppUser.getAuthorities(): role 'CUSTOMER' becomes authority 'ROLE_CUSTOMER'
return List.of(new SimpleGrantedAuthority("ROLE_" + role));
```

5.2 Role rules (URL level)

```
http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/", "/slots", "/login", "/error", "/403", "/css/**",
                         "/api/**", "/actuator/health", "/h2-console/**").permitAll()
        .requestMatchers("/slots/*/book", "/my/**").hasRole("CUSTOMER")
        .requestMatchers("/provider/**").hasRole("PROVIDER")
        .anyRequest().authenticated())
    .formLogin(form -> form.loginPage("/login").usernameParameter("email") ...)
    // wrong role -> 403 page
    .exceptionHandling(ex -> ex.accessDeniedPage("/403"));
```

Request	Anonymous	Customer	Provider
`GET /`, `GET /slots`	200	200	200
`GET/POST /slots/{id}/book`, `/my/**`	302 → /login	200	**403**
`/provider/**`	302 → /login	**403**	200

5.3 Ownership rules (row level)

A role check alone is not enough: Alice is a customer, but she must not cancel Bob's appointment. The service layer therefore checks ownership against the database row and throws

`ForbiddenException`, which becomes 403:

- **Cancel:** `appointment.customer_id` must equal the logged-in user.
- **Confirmation page:** only the appointment's customer, or the provider of its slot, may view it.
- **Remove slot:** `slot.provider_id` must equal the logged-in provider.

CSRF: every state-changing POST requires Spring Security's CSRF token, which Thymeleaf inserts automatically. A forged cross-site POST is rejected with 403 (tested).

6. Enterprise Behavior

6.1 Input validation

Input	Rule	Mechanism
Booking notes	At most 500 characters	`@Size(max = 500)` on `BookingForm`, with `@Valid`
New slot	Service, date and start time are required; the date cannot be in the past	`@NotNull`, `@FutureOrPresent` on `SlotForm`
New slot	Start time in the future; known service; no overlap	`ProviderSlotService` → `BadRequestException` (400) / `SlotConflictException` (409)
Path / query values	Must be the right type (e.g. `/slots/abc/book`)	`MethodArgumentTypeMismatchException` → 400
Page size	Clamped to 1..50, page ≥ 0	`SlotSearchCriteria`

Invalid form input re-displays the form with the error message next to the field and HTTP status **400**.

6.2 Global exception handling

Services throw meaningful exceptions. Two `@ControllerAdvice` classes translate them in one place: `GlobalExceptionHandler` renders the HTML `error.html` page, and `ApiExceptionHandler` returns RFC 9457 `ProblemDetail` JSON for `/api/**`.

Exception	Status	Example
`BadRequestException`, validation, type mismatch	**400**	Notes too long; slot start in the past; `/slots/abc/book`
`ForbiddenException`, Spring Security access denied	**403**	Cancelling someone else's appointment; customer opening `/provider/slots`
`NotFoundException`	**404**	Slot 9999; a removed slot

`SlotConflictException`	**409**	Slot just booked by someone else; already cancelled; overlapping slot
anything else	500	Logged with its stack trace on the server; the user sees a generic message

No stack traces reach the client. The handlers send only the status and a user-facing message, and `application.properties` sets `server.error.include-stacktrace=never`, `include-message=never` and `include-exception=false` for Spring Boot's fallback error page. Tests assert that error responses contain no exception text.

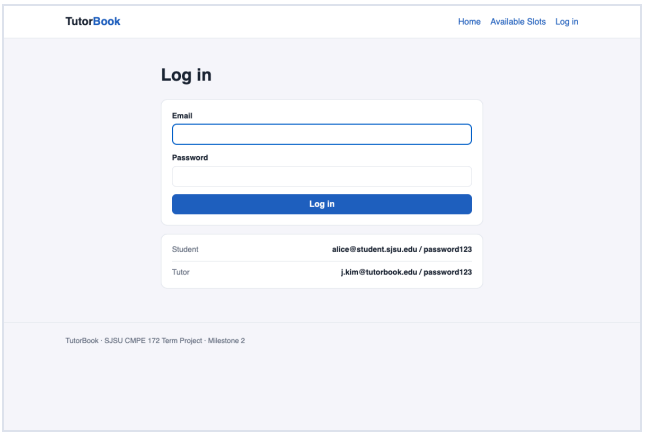
```
GET /api/slots/2  →  404
{"detail":"Slot 2 is not available","instance":"/api/slots/2","status":404,"title":"Not Found"}
```

7. Testing

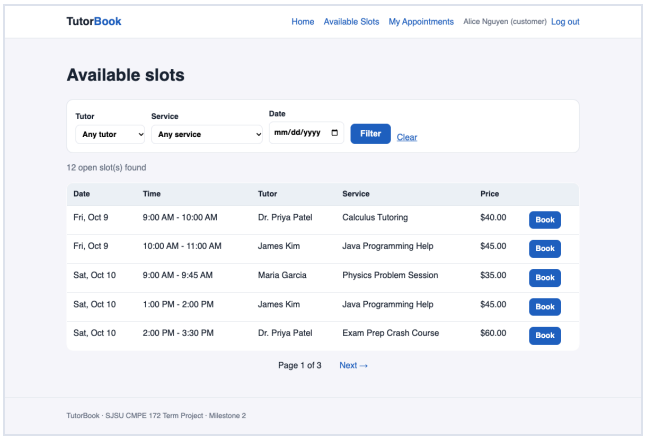
61 tests, all passing (`./mvnw test`).

Test class	Type	#	What it proves
ConcurrentBookingTest	concurrency	6	2 threads book the same slot at the same instant: exactly one succeeds, the other gets 409, and the DB has one BOOKED row (run on 5 slots). Also 8 threads on one slot: exactly 1 success.
BookingTransactionTest	unit (Mockito)	6	Books an open future slot; already-booked → 409; past slot → 409; missing or removed → 404; version mismatch → retry signal; unique-constraint backstop → 409.
BookingServiceTest	unit	3	Retries after a version conflict and succeeds; gives up with 409 after 3 attempts; never retries a real conflict.
AppointmentServiceTest	unit	8	Owner-only cancel (403 for others), cancel twice → 409, started → 409, lost cancel race → 409, unknown → 404, view permissions, upcoming/history split.
ProviderSlotServiceTest	unit	6	End time from service duration, past start → 400, overlap → 409, cannot remove another provider's slot or a booked slot.
FormValidationTest	unit	3	Bean Validation rules on `BookingForm` and `SlotForm`.
SecurityRbacTest	integration	7	BCrypt hashes in DB; correct login → session with ROLE_CUSTOMER; wrong password fails; anonymous → /login; customer → provider pages 403; provider → customer pages 403; missing CSRF → 403.
BookingFlowWebTest	integration	12	Every feature end-to-end through MockMvc: book, 409 on a taken slot, 400 invalid input, 404 without a stack trace, my appointments with COMPLETED, cancel, 403 on someone else's appointment, provider create/remove/overlap, provider bookings.
WebEndpointsTest, DoubleBookingGuardTest	integration	10	Public pages, pagination, filters, JSON ProblemDetail errors, health; the unique backstop and re-booking after cancel.

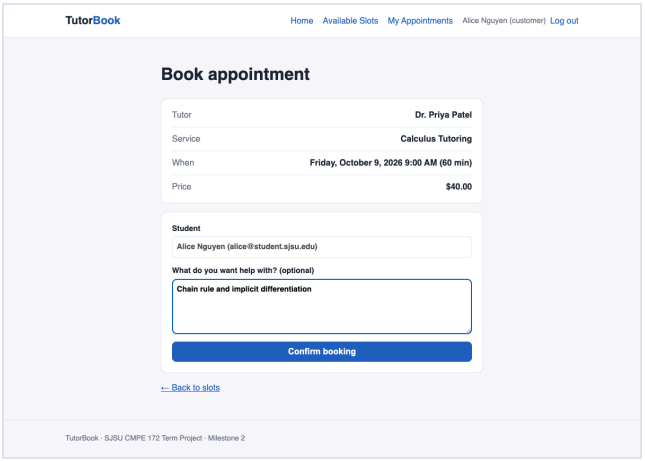
8. Screenshots



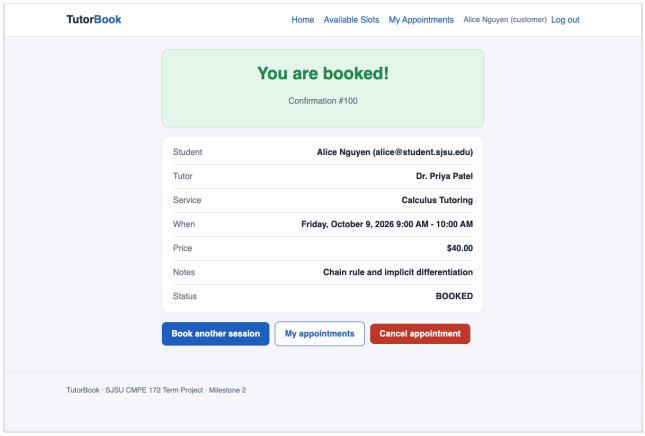
Login



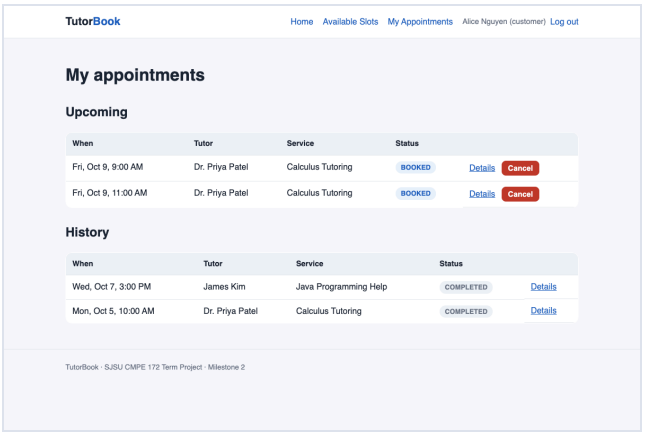
Available slots (logged in as a customer)



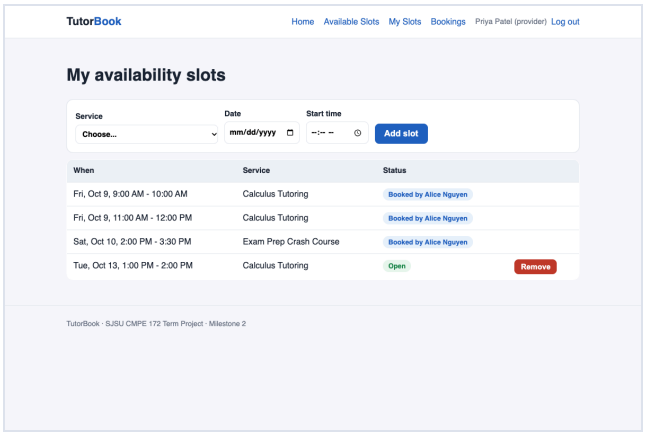
Book appointment form



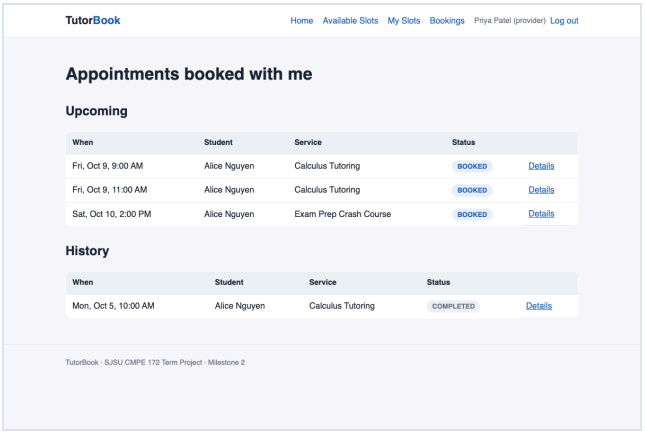
Confirmation, with a cancel button for the owner



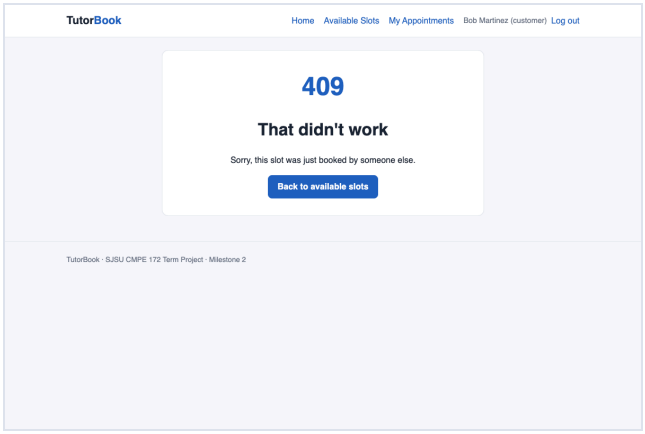
My appointments: upcoming and history (COMPLETED)



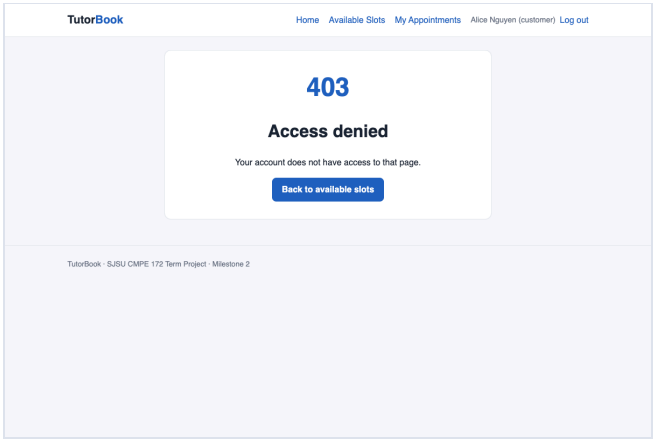
Provider: create / remove slots



Provider: appointments booked with me



409: Bob tried to book a slot Alice just took



403: a customer opening a provider page

9. How to Run

```
git clone https://github.com/samuel1gg/cmpe172-tutorbook.git
cd cmpe172-tutorbook
./mvnw spring-boot:run          # http://localhost:8080
./mvnw test                     # 61 tests
```

Customer login: `alice@student.sjsu.edu` / `password123`. Provider login: `j.kim@tutorbook.edu` / `password123`. Only JDK 17+ is required, since H2 runs in memory and the Maven wrapper downloads Maven.